

robust_safe_rl — Full Mathematical Overview

A file-by-file mathematical walkthrough of the package. The system has three layers:

- `core/` — simulates a quadrotor and controls it (physics + base policy).
- `ood/` — detects when the situation is outside the training distribution.
- `scripts/` — runnable entry points that train and stress-test the detectors.

Motivation: a residual RL controller improves on the base geometric controller under disturbances, but degrades when the state leaves the training distribution. The aim is to **detect** that OOD condition so the residual can be **gated** (disabled / fall back to the base controller) outside its valid domain.

`core/` — physics and control

`core/so3.py` — rotation math

The attitude is a rotation matrix

$$R \in SO(3) = \{R \in \mathbb{R}^{3 \times 3} : R^\top R = I, \det R = 1\}.$$

This file holds the Lie-group operations everything else depends on.

Hat map $\widehat{(\cdot)} : \mathbb{R}^3 \rightarrow \mathfrak{so}(3)$ turns a vector into a skew-symmetric matrix so a cross product becomes a matrix multiply:

$$\hat{\omega} = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}, \quad \hat{\omega} v = \omega \times v.$$

Vee map $(\cdot)^\vee : \mathfrak{so}(3) \rightarrow \mathbb{R}^3$ is the inverse, pulling the vector back out of a skew matrix.

Projection onto $SO(3)$ cleans up a matrix that has drifted from being a true rotation (numerical integration slowly breaks orthonormality). Via SVD $R = U\Sigma V^\top$:

$$R_{\text{proj}} = UV^\top,$$

and if $\det(UV^\top) = -1$ the last column of U is flipped to force $\det = +1$ (a proper rotation, not a reflection).

Rotation error between current R and desired R_d :

$$e_R = \frac{1}{2} (R_d^\top R - R^\top R_d)^\vee.$$

This is the standard geometric attitude error; it lives in $\mathbb{R}^3$ and vanishes exactly when $R = R_d$.

Logarithm map $\log : SO(3) \rightarrow \mathfrak{so}(3)$. With rotation angle θ where $\cos \theta = \frac{1}{2}(\text{tr}(R) - 1)$:

$$\log(R) = \frac{\theta}{2 \sin \theta} (R - R^\top), \quad \theta = \arccos\left(\frac{\text{tr}(R) - 1}{2}\right).$$

- `so3_log` returns the matrix form.
- `so3_log_vector` returns the rotation vector (axis $\times$ angle) $\phi = \theta a$, with a **numerically stable branch near $\theta \rightarrow \pi$** , where $\sin \theta \rightarrow 0$ makes the naive formula blow up. There it extracts the axis from the diagonal of $\frac{1}{2}(R + I)$. This matters because the feature extractor encodes relative attitude as a log and disturbances can produce large angles.

Normalize returns $v/\|v\|$, or a fallback direction if $\|v\|$ is near zero (prevents division-by-zero when building desired axes).

`core/dynamics.py` — the quadrotor plant

Integrates the rigid-body equations of motion in a **North-East-Down (NED)** frame. State is (x, v, R, ω) : position, velocity, attitude, body-frame angular velocity. The continuous dynamics are

$$\begin{aligned} \dot{x} &= v, \\ \dot{v} &= g e_3 - \frac{f}{m} R e_3 + \frac{1}{m} f_{\text{ext}}, \\ \dot{R} &= R \hat{\omega}, \\ \dot{\omega} &= J^{-1}(M - \omega \times J \omega), \end{aligned}$$

where $e_3 = [0, 0, 1]^\top$, m is mass, J the inertia matrix, g gravity, $f \in \mathbb{R}$ the scalar thrust, $M \in \mathbb{R}^3$ the body moment, and f_{ext} an external force disturbance.

NED convention. Gravity is $+g e_3$ (down is positive z), and thrust enters as $-\frac{f}{m} R e_3$ because the body thrust axis points *up* against the down-axis. This is why a height of 1 m above the origin is written $z = -1$.

Disturbance modes (`sample_external_force`) — this is what makes an episode in- or out-of-distribution:

$$\begin{aligned} \text{mode 0 : } & f_{\text{ext}} = 0 \\ \text{mode 1 (ID) : } & f_{\text{ext}} \sim \mathcal{U}(-3, 3)^3 \\ \text{mode 2 (loose OOD) : } & f_{\text{ext}} \sim \mathcal{U}(-5, 5)^3 \\ \text{mode 3 (strict OOD) : } & f_{\text{ext}} \sim \mathcal{U}(-5, 5)^3 \text{ s.t. } \max_i |f_{\text{ext},i}| > 3. \end{aligned}$$

Mode 1 is the training domain (each axis within ± 3 N). Mode 3 forces at least one axis *outside* that box — the "subtle, same-type" OOD the autoencoder struggles with. By default the force is drawn once at reset and held constant for the episode (`(force_sample_each_step=False)`).

Integration is fixed-step **RK4**. Stacking the state into $y = [x, v, \text{vec}(R), \omega] \in \mathbb{R}^{18}$ and calling the right-hand side $\Phi(y)$:

$$\begin{aligned} k_1 &= \Phi(y), & k_2 &= \Phi(y + \tfrac{h}{2} k_1), \\ k_3 &= \Phi(y + \tfrac{h}{2} k_2), & k_4 &= \Phi(y + h k_3), \\ y_{\text{next}} &= y + \tfrac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4), & h &= dt. \end{aligned}$$

After each step R is re-projected onto $SO(3)$ to undo integration drift. Mass m and inertia J are constructor arguments — the hook the mass/MOI OOD experiment uses (scaling them while keeping the force in-distribution).

`core/controller.py` — geometric SE(3) tracking controller

The **base controller** (the nominal policy the residual RL will sit on top of), adapted to NED. Given state (x, v, R, ω) and desired (x_d, v_d, a_d, b_{1_d}) :

Position and velocity errors:

$$e_x = x - x_d, \quad e_v = v - v_d.$$

Desired force vector (PD law + gravity compensation + acceleration feedforward), with diagonal gains $k_x = 18m I, k_v = 10m I$ (mass-scaled):

$$A = k_x e_x + k_v e_v + mg e_3 - m a_d.$$

Total thrust is the projection of A onto the current body thrust axis Re_3 , clamped nonnegative:

$$f = \max(0, A \cdot (Re_3)).$$

Desired attitude. The body z -axis aligns with A ; the desired heading b_{1_d} fixes the remaining yaw:

$$b_{3_d} = \frac{A}{\|A\|}, \quad b_{2_d} = \frac{b_{3_d} \times b_{1_d}}{\|b_{3_d} \times b_{1_d}\|}, \quad b'_{1_d} = b_{2_d} \times b_{3_d},$$

$$R_d = \text{proj}_{SO(3)} [b'_{1_d} \mid b_{2_d} \mid b_{3_d}].$$

Desired angular velocity and its derivative come from *finite-differencing* R_d across steps (no analytic $\dot{R}_d$ is available):

$$\omega_d = \frac{1}{dt} \left(\log(R_{d,\text{prev}}^\top R_d) \right)^\vee, \quad \dot{\omega}_d = \frac{\omega_d - \omega_{d,\text{prev}}}{dt}.$$

On the first step after a reset these are zero (no history yet), so the first one or two samples have zero feedforward.

Attitude and angular-velocity errors:

$$e_R = \frac{1}{2} (R_d^\top R - R^\top R_d)^\vee, \quad e_\Omega = \omega - R^\top R_d \omega_d.$$

Control moment (attitude PD + gyroscopic term + feedforward), with $k_R = 8.81$, $k_\Omega = 2.54$:

$$M = -k_R e_R - k_\Omega \omega_d + J \dot{\omega}_d$$

- $J \left(\hat{\omega}_d R^\top R_d \omega_d - R^\top R_d \dot{\omega}_d \right)$.

The outputs (f, M) feed straight into `Dynamics.step`. Empirically this tracks the reference circle to about 5 mm steady-state.

`core/trajectory.py` — reference trajectory

A circle at constant altitude with zero yaw. With radius r , speed s , angular rate $w = s/r$:

$$x_d(t) = \begin{bmatrix} r \sin(wt) \\ -r \cos(wt) + r \\ z_0 \end{bmatrix}, \quad v_d(t) = \begin{bmatrix} rw \cos(wt) \\ rw \sin(wt) \\ 0 \end{bmatrix}, \quad a_d(t) = \begin{bmatrix} -rw^2 \sin(wt) \\ rw^2 \cos(wt) \\ 0 \end{bmatrix}.$$

v_d and a_d are the exact analytic derivatives of x_d (used as feedforward in the controller). The desired heading is constant, $b_{1_d} = [1, 0, 0]^\top$ (body x -axis points North). The $+r$ offset in the second component starts the circle at the origin. Default $z_0 = -1 \rightarrow 1$ m altitude.

`ood/shared/` — detector-agnostic building blocks

Both detectors reuse everything here.

`ood/shared/features.py` — the residual feature

The conceptual heart of the approach. To detect a disturbance, compare **two rollouts driven by the same command**: a *true* system (with disturbance) and a *nominal* system (no disturbance, nominal parameters). The discrepancy between them is the disturbance signature.

The command applied to both at step t is the controller's command computed from the **true** state, and the feature pairs *that action* with the **state discrepancy at $t + 1$ **. This "transition alignment" ensures the feature captures the disturbance's effect, not a mismatch between two different controllers.

Each **16-D step feature** is

$$\phi_t = [e_x, e_v, e_R^{\text{vec}}, e_\omega, u] \in \mathbb{R}^{16},$$

with sign convention *nominal minus true*:

$$\begin{aligned} e_x &= x_{t+1}^{\text{nom}} - x_{t+1}^{\text{true}}, & e_v &= v_{t+1}^{\text{nom}} - v_{t+1}^{\text{true}}, \\ e_R^{\text{vec}} &= \log \left(R_{t+1}^{\text{true}\top} R_{t+1}^{\text{nom}} \right)^\vee, & e_\omega &= \omega_{t+1}^{\text{nom}} - \omega_{t+1}^{\text{true}}, \end{aligned}$$

and $u = [f, M_x, M_y, M_z]$ the commanded action. The relative attitude is encoded as a rotation vector via the stable log map.

A full **sample** stacks H consecutive step features (history length H , default 10):

$$\Phi = [\phi_{t-H+1}, \dots, \phi_t] \in \mathbb{R}^{16H}.$$

`feature_metadata` records `feature_version=2`, `history_len`, dims, and conventions into each checkpoint, so test/eval scripts can assert compatibility.

`ood/shared/dataset.py` — rollout collection

Turns simulations into datasets of samples Φ . Two collectors, both running the true/nominal pair described above:

`collect_dataset` varies the **force** disturbance. It runs `num_episodes` rollouts under a chosen force mode, and every time the history buffer fills to H it emits a stacked sample Φ (plus the true force that produced it). Training uses mode 1 (ID); OOD evaluation uses mode 3 (strict OOD).

`collect_parameter_dataset` keeps forces in-distribution but scales **mass and inertia** per episode:

$$m_{\text{true}} = \sigma_m m_{\text{nom}}, \quad J_{\text{true}} = J_{\text{nom}} \text{diag}(\sigma_J),$$

with nominal reference $m_{\text{nom}} = 2.0$, $J_{\text{nom}} = \text{diag}(0.022, 0.022, 0.04)$. The nominal rollout always uses nominal parameters, so the feature isolates the parameter mismatch. This is the "different disturbance type" the autoencoder *can* detect.

Standardization. Fit mean/std on training data and apply to all splits:

$$\mu = \text{mean}(\Phi_{\text{train}}), \quad \sigma = \max(\text{std}(\Phi_{\text{train}}), 10^{-6}), \quad \tilde{\Phi} = \frac{\Phi - \mu}{\sigma}.$$

`standardize` does train/val/OOD together; `apply_standardization` reuses stored (μ, σ) at test time (with dimension checks). The clamp on σ prevents division by zero on near-constant features.

`ood/shared/metrics.py` — scoring and detection metrics

Everything operates on the per-sample **anomaly score** = reconstruction MSE.

`evaluate_errors` runs the model and returns, per sample i , the reconstruction error and latent code:

$$\text{err}_i = \frac{1}{d} \|\hat{\Phi}_i - \Phi_i\|^2, \quad z_i = \text{encoder}(\Phi_i).$$

`detection_report` derives thresholds purely from the ID error distribution — the max, the 95/99/99.5 percentiles, and mean + 3σ :

$$\tau_{\max} = \max_i \text{err}_i^{\text{ID}}, \quad \tau_{p99} = \text{percentile}_{99}, \quad \tau_{3\sigma} = \mu_{\text{ID}} + 3\sigma_{\text{ID}}.$$

For each threshold it reports the **false-alarm rate** $\Pr(\text{err}^{\text{ID}} > \tau)$ and **detection rate** $\Pr(\text{err}^{\text{OOD}} > \tau)$. This is exactly the "flag OOD when reconstruction error significantly exceeds the ID outlier error" rule.

`binary_curves` computes threshold-free curves. Sorting all scores descending and sweeping the threshold gives the ROC curve (FPR, TPR) and its area, plus precision/recall and average precision:

$$\text{AUROC} = \int_0^1 \text{TPR} d(\text{FPR}), \quad \text{AP} = \frac{1}{|\mathcal{P}|} \sum_{i \in \mathcal{P}} \text{precision}_i.$$

(This is the file where the `(np.trapz) → (np.trapezoid)` NumPy-2 crash was fixed.)

`compute_confusion_matrix` / `threshold_metrics` give the confusion counts (TN, FP, FN, TP) at fixed thresholds and derived rates including balanced accuracy $\frac{1}{2} \left(\frac{TN}{TN+FP} + \frac{TP}{TP+FN} \right)$. `summarize` is descriptive stats (mean/median/percentiles/max) of a score array.

`ood/shared/plots.py` — diagnostics

Renders the standard figures (headless, Agg backend): the **reconstruction-error histogram** (ID vs OOD with the p_{99} threshold line), the **sorted-error curve**, and **latent-space scatter** in 2-D (all coordinate pairs) and 3-D. These are the visual version of the two detection signals — error separation and latent-cluster separation.

`ood/shared/io_utils.py` — checkpoint bookkeeping

Small helpers. `get_next_indexed_path` finds the next free `(NNN_)` prefix so repeated runs auto-increment instead of overwriting; `get_checkpoint_by_index` loads a specific one. No math.

`ood/ood_autoencoder/` — Detector 1 (implemented)

`ood/ood_autoencoder/autoencoder.py` — the model

A symmetric MLP autoencoder. Encoder E maps input to a low-dimensional latent, decoder D maps back:

$$z = E(\Phi) \in \mathbb{R}^\ell, \quad \hat{\Phi} = D(z), \quad \ell \ll 16H.$$

Hidden layers are `hidden_dims` with a chosen activation (ReLU/Tanh/ELU/GELU); the decoder mirrors the encoder. Training minimizes reconstruction MSE:

$$\mathcal{L}_{\text{rec}} = \frac{1}{d} \|D(E(\Phi)) - \Phi\|^2.$$

The detection idea: trained only on ID data, the autoencoder reconstructs ID features well (small error) but unfamiliar OOD features poorly (large error). So reconstruction error doubles as an anomaly score, and the latent z gives a second view — ID points cluster, OOD points ideally sit apart. `reconstruction_error` returns per-sample MSE + latent; `save`/`load` persist the weights together with (μ, σ) , thresholds, and feature metadata.

Empirical finding (documented in the code): this works for mass/MOI OOD (a *different* disturbance type) but **fails on strict force OOD** (same type, just out of range) — the ID and OOD error distributions overlap too much. That negative result is the motivation for Detector 2.

`ood/ood_autoencoder/regularization.py` — optional latent penalties

Two penalties can be added to sharpen the latent manifold and (hopefully) widen the ID/OOD gap.

Sparse — L1 activity penalty on the latent code, pushing each sample to use few coordinates:

$$\mathcal{L}_{\text{sparse}} = \frac{1}{\ell} \sum_j |z_j|.$$

Contractive — penalizes the sensitivity of the latent to input perturbations, i.e. the Frobenius norm of the encoder Jacobian, estimated cheaply with a Hutchinson (random-probe) estimator on a subset of the batch:

$$\mathcal{L}_{\text{contractive}} = \left\| \frac{\partial z}{\partial \Phi} \right\|_F^2 \approx \mathbb{E}_\epsilon [\|\nabla_\Phi(z^\top \epsilon)\|^2], \quad \epsilon \in \{-1, +1\}^\ell,$$

divided by ℓ so the coefficient is decoupled from bottleneck width. Total loss is $\mathcal{L}_{\text{rec}} + \lambda \mathcal{L}_{\text{reg}}$.

`ood/ood_residual/` — Detector 2 (implemented)

Predicts the next physical residual from a history window and flags OOD when the realized residual departs from the prediction. Reuses `core/` and `ood/shared/`.

`ood/ood_residual/dataset.py` — windowed supervised pairs

Emits `(history window, next residual)` pairs. Input is H stacked 16-D step features $\Phi \in \mathbb{R}^{16H}$; target is the next physical residual

$$\delta_{t+1} = [e_x, e_v, e_R, e_\omega]_{t+1} \in \mathbb{R}^{12}$$

(the non-action part of ϕ_{t+1}), or the next Δ of them for multi-horizon. `target_dim=6` predicts only the translational part $[e_x, e_v]$. `standardize_io` fits separate mean/std on inputs and targets from training data.

`ood/ood_residual/predictor.py` — Gaussian MLP

An MLP with two heads producing a diagonal Gaussian over the residual:

$$(\mu_\theta, \log \sigma_\theta^2) = f_\theta(\Phi), \quad \hat{\delta}_{t+1} \sim \mathcal{N}(\mu_\theta, \text{diag}(\sigma_\theta^2)).$$

Trained by Gaussian negative log-likelihood (per output dim, summed):

$$\mathcal{L} = \sum_d \left[\frac{1}{2} \left(\frac{(\delta_d - \mu_{\theta,d})^2}{\sigma_{\theta,d}^2} + \log \sigma_{\theta,d}^2 \right) \right]$$

- $\log \sigma_{\theta,d}^2$

The first term is MSE scaled by predicted variance; the second stops the model from claiming huge σ everywhere. Together they calibrate the uncertainty. `log_var` is clamped to a sane range for numerical stability.

`ood/ood_residual/scoring.py` — scores, CUSUM, detector interface

Per-sample anomaly scores from the predictor:

$$\text{zscore} = \text{mean}_d \frac{|\delta_d - \mu_{\theta,d}|}{\sigma_{\theta,d}}, \quad \text{nll} = \mathcal{L}_{\text{NLL}}, \quad \text{mse} = \text{mean}_d (\delta_d - \mu_{\theta,d})^2.$$

`zscore` is the calibrated surprise (how many σ off), the recommended default. A one-sided **CUSUM** turns the per-step score stream into a detection statistic that catches small persistent shifts (a constant force just outside the training range):

$$S_t = \max(0, S_{t-1} + (s_t - k)), \quad \text{flag OOD when } S_t > \tau,$$

with slack k (default: ID score mean) and threshold τ calibrated on held-out ID episodes.

`ResidualDetector` wraps a checkpoint into the shared `score(...)` contract the future gating layer depends on.

scripts/ — runnable entry points

- `run_tracking_demo.py` — closed-loop sanity check. Runs controller + dynamics on the reference circle and plots trajectory and tracking error $\|x - x_d\|$. No learning; confirms the base stack works.
- `train_autoencoder.py` — the training pipeline. Collects ID train/val (mode 1) and strict-ODD (mode 3), standardizes, trains by minimizing $\mathcal{L}_{\text{rec}}$ with AdamW and early-stopping on validation MSE, then reports detection rates at each ID threshold and saves the checkpoint + plots.
- `test_autoencoder.py` — reloads a checkpoint and re-tests on *fresh* ID/ODD rollouts, comparing stored thresholds against fresh ones (checks generalization to unseen episodes).
- `evaluate_ood_detection.py` — full confusion-matrix report for **force** OOD at the τ_{max} threshold: accuracy, false-alarm, detection, miss rates.
- `evaluate_mass_moi_ood.py` — the **cross-type** test. Evaluates a force-trained detector on mass/MOI OOD, computing AUROC/AP, per-episode error vs. a severity measure

$$\text{severity} = \max(|\sigma_m - 1|, \max_i |\sigma_{J,i} - 1|),$$

and extensive latent plots (including ID-scale-zoom and symlog views, since parameter-ODD latents can land far from the ID cluster). This is where the autoencoder actually succeeds.

- `sweep_autoencoder.py` — grid sweep over architecture, latent dim, and regularizers. Collects one shared dataset, trains each config, writes per-run checkpoints/metrics/history and a master summary.
- `train_residual.py` — trains the Gaussian residual-dynamics detector. Collects ID/strict-ODD windowed pairs, standardizes inputs and targets, minimizes NLL with early stopping, reports AUROC/AP and per-window detection at ID thresholds, saves checkpoint + plots.
- `evaluate_residual_detection.py` — clear summary for the residual detector: (A) per-window AUROC/AP and confusion at ID thresholds, and (B) per-episode **CUSUM** detection rate, ID false-alarm rate, and mean detection delay. Writes a JSON summary.

The whole picture

1. `core/` produces the true-vs-nominal discrepancy under disturbances.
2. `features.py` packages it into the transition-aligned residual Φ .
3. `dataset.py` turns rollouts into ID/ODD datasets.
4. The autoencoder scores anomalies by reconstruction error and latent position.

5. The scripts train and stress-test it.

Documented result: reconstruction/latent detection catches *parameter*-type OOD (mass/MOI) but **not** subtle *same-type force* OOD — which is exactly what the planned residual-dynamics detector (ood_residual) is meant to fix.